Agricultural Commodity Trading Platform

A comprehensive full-stack application for managing agricultural commodity trading between farmers and buyers, featuring real-time market price tracking, contract management, and payment processing. The platform ensures transparent and efficient transactions in the agricultural supply chain.

■ Now Available in Two Backend Options:

- **Node.js/Express** (TypeScript) - Original implementation
- **Java Spring Boot** - Enterprise-grade migration with enhanced performance

■ Features

Core Functionality

- **Multi-role User Management**: Role-based authentication (Farmer, Buyer, Admin)
- **Smart Contract Management**: Create, negotiate, and track agricultural contracts
- **Contract Request System**: Buyers can send requests to farmers, farmers can accept/reject
- **5-Star Rating & Feedback**: Buyers can rate farmers after contract completion
- **Real-time Market Price Tracking**: Live price feeds and historical trend analysis
- **Integrated Payment System**: Transaction management and payment tracking
- **Automated Cash Payment Completion**: Cash payments automatically marked as COMPLETED
- **Analytics Dashboard**: Comprehensive statistics and business insights
- **Location-based Pricing**: Geographic price variations and delivery management
- **Bilingual Support**: Full English and Kannada language support for wider accessibility
- **Enhanced Admin Controls**: Advanced transaction management with status toggling and deletion capabilities
- **Admin User Management**: Permanent user deletion with complete data cleanup
- **Bangalore-Specific Locations**: Focused location selection with automated daily date updates for 20 areas
- **Automated Contract Completion**: Contracts automatically marked as COMPLETED when all payments are processed
- **Improved Transaction Visibility**: Buyers can see all transactions related to their contracts, not just those they created
- **Admin-Only Contract Status Changes**: Only admins can change COMPLETED contracts back to other statuses
- **Glass Morphism UI**: Modern transparent glass-effect login interface

Key Benefits

- ■ **Market Transparency**: Real-time pricing data for informed decision-making
- ■ **Risk Mitigation**: Fixed-price contracts protect against market volatility
- ■ **Direct Trade**: Eliminate middlemen for better profit distribution
- ■ **Efficient Operations**: Streamlined digital workflow for all stakeholders

- ■ **Data-driven Insights**: Analytics for strategic planning
- ■ **Multilingual Support**: Accessible in both English and Kannada for broader reach
- ■ **Enhanced Admin Controls**: Advanced transaction management capabilities
- ■ **Automated Cash Payments**: Cash transactions instantly marked as completed
- ■ **Localized Delivery**: Bangalore-area-specific location selection for improved logistics
- ■ **Automated Date Updates**: Daily automatic date updates for all 20 Bangalore areas
- ■ **Automated Contract Completion**: Contracts automatically marked as COMPLETED when all payments are processed
- ■ **Complete Transaction Visibility**: Buyers can see all transactions related to their contracts
- ■ **Secure Status Management**: Only admins can change completed contract statuses
- ■ **Complete User Deletion**: Admins can permanently delete users with all associated data
- ■ **Modern Glass UI**: Beautiful transparent glass-morphism login interface
- ■ **Flexible Storage Options**: PostgreSQL for production or JSON file storage for development/testing
- ■ **Two-Way Contract Creation**: Farmers create direct contracts, buyers send requests
- ■ **Verified Rating System**: Only buyers with completed contracts can rate farmers

■■ Tech Stack

Backend Options

Option 1: Node.js/Express (TypeScript)
- **Node.js** with **Express.js**
- **TypeScript** for enhanced type safety
- **Prisma ORM** for robust database management
- **PostgreSQL** for reliable data persistence (Production)
- **JSON File Database** for simple setup and portability (Development/Testing)
- **JWT** for secure authentication
- **BcryptJS** for secure password hashing
- **Zod** for schema validation
- **Express Validator** for request validation

Option 2: Java Spring Boot ■ NEW
- **Java 17** with **Spring Boot 3.2.0**
- **Spring Security** with JWT authentication
- **JPA/Hibernate** for database ORM
- **PostgreSQL** for production database
- **Maven** for build automation
- **Lombok** for reduced boilerplate code
- **Jackson** for JSON processing

- **Spring Validation** for request validation
- **Enterprise-grade security** and performance

Frontend
- **React 18** with **TypeScript** for type-safe development
- **Vite** for lightning-fast development builds
- **React Router** for dynamic client-side routing
- **TanStack Query** for advanced data fetching and caching
- **Tailwind CSS** for responsive, utility-first styling
- **Recharts** for interactive data visualization
- **Lucide React** for consistent iconography
- **React Hook Form** for form management
- **Date-FNS** for date manipulation
- **React-i18next** for multilingual support (English/Kannada)

Database Schema
- **User**: Farmers, Buyers, and Admin accounts with role-based access, ratings, and profiles
- **Product**: Agricultural commodities with categorization
- **Contract**: Structured agreements with lifecycle management
- **MarketPrice**: Real-time pricing with geographic variations
- **Transaction**: Payment tracking with multiple payment methods
- **ContractRequest**: Buyer-initiated requests awaiting farmer approval
- **FarmerRating**: 5-star ratings and feedback from buyers

■ Project Structure

```
agricultural-commodity-platform/
■■■ client/ # Frontend React application
■ ■■■ src/
■ ■ ■■■ components/ # Reusable UI components
■ ■ ■ ■■■ Header.tsx
■ ■ ■ ■■■ Layout.tsx
■ ■ ■ ■■■ PrivateRoute.tsx
■ ■ ■ ■■■ Sidebar.tsx
■ ■ ■ ■■■ ... more components
■ ■ ■■■ contexts/ # React Context providers
■ ■ ■ ■■■ AuthContext.tsx
■ ■ ■■■ pages/ # Page components
■ ■ ■ ■■■ ContractDetail.tsx
```

■ ■ ■ ■■■ Contracts.tsx
■ ■ ■ ■■■ CreateContract.tsx
■ ■ ■ ■■■ Dashboard.tsx
■ ■ ■ ■■■ Login.tsx
■ ■ ■ ■■■ MarketPrices.tsx
■ ■ ■ ■■■ Profile.tsx
■ ■ ■ ■■■ Register.tsx
■ ■ ■ ■■■ SendRequest.tsx # Buyer contract request page
■ ■ ■ ■■■ Transactions.tsx
■ ■ ■ ■■■ Users.tsx
■ ■ ■ ■■■ ContractRequests.tsx # Farmer request management
■ ■ ■■■ types/ # TypeScript type definitions
■ ■ ■ ■■■ index.ts
■ ■ ■■■ utils/ # JavaScript utility modules
■ ■ ■ ■■■ apiUtils.js
■ ■ ■ ■■■ chartUtils.js
■ ■ ■ ■■■ contractUtils.js
■ ■ ■ ■■■ dataAnalytics.js
■ ■ ■ ■■■ dateUtils.js
■ ■ ■ ■■■ i18n.ts
■ ■ ■ ■■■ index.js
■ ■ ■ ■■■ jsUtils.js
■ ■ ■ ■■■ localization.js
■ ■ ■ ■■■ marketPriceAnalysis.js
■ ■ ■ ■■■ notificationUtils.js
■ ■ ■ ■■■ validationUtils.js
■ ■ ■■■ lib/ # Utility libraries
■ ■ ■ ■■■ queryClient.ts
■ ■ ■■■ App.tsx # Main application component
■ ■ ■■■ main.tsx # Application entry point
■ ■ ■■■ index.css # Global styles
■ ■■■ public/
■ ■■■ package.json
■■■ server/ # Backend API server (Node.js/Express)
■ ■■■ src/
■ ■ ■■■ routes/ # API route handlers
■ ■ ■ ■■■ auth.ts
■ ■ ■ ■■■ contracts.ts

■ ■ ■ ■■■ contractRequests.ts # Contract request endpoints
■ ■ ■ ■■■ marketPrices.ts
■ ■ ■ ■■■ products.ts
■ ■ ■ ■■■ ratings.ts # Rating system endpoints
■ ■ ■ ■■■ transactions.ts
■ ■ ■ ■■■ users.ts
■ ■ ■■■ middleware/ # Express middleware
■ ■ ■ ■■■ auth.ts
■ ■ ■■■ utils/ # Utility functions
■ ■ ■ ■■■ jwt.ts
■ ■ ■■■ server.ts # Main server file
■ ■ ■■■ seed.ts # Data seeding script
■ ■ ■■■ addSample*.ts # Sample data addition scripts
■ ■ ■■■ update*.ts # Data update scripts
■ ■■■ prisma/
■ ■ ■■■ schema.prisma # Database schema definition
■ ■ ■■■ migrations/ # Database migration files
■ ■■■ package.json
■■■ server-java/ # Backend API server (Java Spring Boot) ■ NEW
■ ■■■ src/main/java/com/agri/trading/
■ ■ ■■■ AgriTradingApplication.java # Main application class
■ ■ ■■■ config/ # Configuration classes
■ ■ ■ ■■■ DataLoaderConfig.java
■ ■ ■ ■■■ JwtAuthFilter.java
■ ■ ■ ■■■ SecurityConfig.java
■ ■ ■■■ controller/ # REST Controllers
■ ■ ■ ■■■ AuthController.java
■ ■ ■ ■■■ ContractController.java
■ ■ ■ ■■■ ContractRequestController.java
■ ■ ■ ■■■ MarketPriceController.java
■ ■ ■ ■■■ ProductController.java
■ ■ ■ ■■■ RatingController.java
■ ■ ■ ■■■ TransactionController.java
■ ■ ■ ■■■ UserController.java
■ ■ ■■■ dto/ # Data Transfer Objects
■ ■ ■ ■■■ AuthResponse.java
■ ■ ■ ■■■ LoginRequest.java
■ ■ ■ ■■■ RegisterRequest.java

■ ■ ■■■ model/ # JPA Entities
■ ■ ■ ■■■ User.java
■ ■ ■ ■■■ Product.java
■ ■ ■ ■■■ Contract.java
■ ■ ■ ■■■ MarketPrice.java
■ ■ ■ ■■■ Transaction.java
■ ■ ■ ■■■ *.java (Enums)
■ ■ ■■■ repository/ # Spring Data Repositories
■ ■ ■ ■■■ UserRepository.java
■ ■ ■ ■■■ ProductRepository.java
■ ■ ■ ■■■ ContractRepository.java
■ ■ ■ ■■■ MarketPriceRepository.java
■ ■ ■ ■■■ TransactionRepository.java
■ ■ ■■■ security/ # JWT Security
■ ■ ■ ■■■ JwtTokenProvider.java
■ ■ ■■■ service/ # Business Logic
■ ■ ■■■ AuthService.java
■ ■■■ src/main/resources/
■ ■ ■■■ application.properties # Application configuration
■ ■■■ pom.xml # Maven dependencies
■ ■■■ README.md # Java backend documentation
■■■ .env # Environment variables
■■■ .gitignore
■■■ package.json # Root package configuration
■■■ run.sh # Setup and execution script
■■■ run-java-backend.sh # Java backend run script
■■■ README.md # Project documentation
■■■ DEPLOYMENT_STEPS.md # Vercel deployment guide
■■■ QUICK_START.md # Quick setup guide
```

**Empowering farmers and buyers with transparent, efficient agricultural trade**

## ■ Quick Start

### Choose Your Backend

This project supports **two backend implementations**:

1. **Node.js/Express (TypeScript)** - Quick setup, flexible development
```

2. **Java Spring Boot** - Enterprise-grade, production-ready

Both backends are **100% compatible** with the same React frontend!

Prerequisites
- **Node.js** (v18 or higher) - For Node.js backend
- **Java 17+** and **Maven 3.6+** - For Java backend
- **PostgreSQL** (v12 or higher) - Optional for file-based database mode
- **Git** for version control

Automated Setup (Node.js Backend)

For the quickest setup with Node.js backend:

```bash
chmod +x run.sh
./run.sh
```

Follow the interactive prompts to set up, seed, and start the application.

Quick Start with Java Backend ■

```bash
chmod +x run-java-backend.sh
./run-java-backend.sh
```

This will build and start the Java Spring Boot backend automatically.

Manual Installation

1. Clone and Navigate to Project

```bash
cd agricultural-commodity-platform
```

2. Install Dependencies

Install all dependencies for root, server, and client:

```bash
npm run install:all
```

3. Database Setup

Option A: PostgreSQL Database (Production)

1. **Create PostgreSQL Database**

```sql
CREATE DATABASE contract_farming;
```

2. **Configure Environment Variables**

Create `.env` file in the `server` directory:

```bash
cd server
cp .env.example .env
```

Edit `server/.env` with your database credentials:

```env
DATABASE_URL="postgresql://username:password@localhost:5432/contract_farming?schema=public"
JWT_SECRET="your-super-secret-jwt-key-change-in-production"
JWT_EXPIRES_IN="7d"
PORT=5000
NODE_ENV=development
USE_FILE_DATABASE=false # Set to false for PostgreSQL
```

3. **Run Database Migrations**

```bash
cd server
npm run prisma:generate
npm run prisma:migrate
```

This will:
- Generate Prisma Client
- Create database tables

- Set up the schema

(Optional) Open Prisma Studio to view/edit database:

```bash
npm run prisma:studio
```

Option B: JSON File Database (Development/Testing)

1. **Enable File Database**

Edit `server/.env`:

```env
USE_FILE_DATABASE=true # Enable file-based storage
FILE_DATABASE_AUTO_SAVE=true # Auto-save changes
FILE_DATABASE_PATH=./config/database.json
```

2. **No Database Setup Required**

The application will use `/server/config/database.json` to store all data.
Sample data is pre-loaded, so you can start immediately!

Benefits of File Database:
- ■ No PostgreSQL installation needed
- ■ Easy setup - just run and go
- ■ Portable - single file contains all data
- ■ Easy backup and restore
- ■ Perfect for development and testing
- ■ Version control friendly

Switch Between Modes:
Simply toggle `USE_FILE_DATABASE` in `.env` between `true` (file) and `false` (PostgreSQL).

4. Seed Initial Data

```bash
cd server
npm run seed
```

This creates:

- Admin user: `admin@contractfarming.com` (see deployment docs for password)
- Sample products (Wheat, Rice, Tomato, Potato, Mango, etc.)
- Sample market prices

5. Start the Application

Option A: Node.js Backend

Development Mode (Runs both server and client):

```bash
npm run dev
```

This starts:
- Backend server on `http://localhost:5000`
- Frontend client on `http://localhost:3000`

Or run separately:

Backend only:
```bash
cd server
npm run dev
```

Frontend only:
```bash
cd client
npm run dev
```

Option B: Java Spring Boot Backend ■

Using the run script:
```bash
./run-java-backend.sh
```

Using Maven directly:
```bash
cd server-java
mvn spring-boot:run
```

```

**Using JAR file**:

```bash
cd server-java
mvn clean package
java -jar target/agri-trading-1.0.0.jar
```

This starts:
- Java backend on `http://localhost:5000`
- Frontend client on `http://localhost:3000` (run separately)

> **Note**: The React frontend works seamlessly with **both** backends without any code changes!

■ Usage

1. Registration

1. Open `http://localhost:3000`
2. Click "Sign up" or navigate to `/register`
3. Choose your role:
- **Farmer**: Can create contracts, track production
- **Buyer**: Can accept contracts, manage purchases
- **Admin**: Full system access, user management
4. Fill in your details and create an account

Default Accounts

Admin Account:
- Email: `admin@contractfarming.com`
- Password: *(Contact system administrator or check deployment configuration)*

Farmer Account:
- Email: `farmer@example.com`
- Password: `password123`

Buyer Account:
- Email: `buyer@example.com`
- Password: `password123`

PostgreSQL Mode:
```

Run the seed script to create default accounts.

You can use these accounts to access the system. For security, consider changing the passwords after first login.

2. Creating a Contract (Farmer)

1. Login as a Farmer
2. Navigate to "Create Contract" from the sidebar
3. Select a buyer
4. Enter contract details:
- Product selection from available commodities
- Quantity and unit of measurement
- Price per unit
- Start and delivery dates
- Delivery location (now with Bangalore-specific area selection)
- Terms and conditions
5. Submit to create the contract

3. Managing Contracts

- **Farmers** can:
- Create new contracts (direct contracts with buyers)
- View incoming contract requests from buyers
- Accept or reject buyer requests
- View all their contracts
- Update contract status (Draft → Pending → Active → Completed)
- Negotiate terms with buyers

- **Buyers** can:
- Send contract requests to farmers (awaiting farmer approval)
- View contracts offered to them
- Approve/reject pending contracts
- Track active contracts
- View all transactions related to their contracts
- Initiate payments
- Rate farmers after contract completion

- **Admins** can:
- View all contracts and requests
- Manage users (create, edit, delete)

- **Permanently delete users with all associated data** (contracts, transactions, ratings, notifications)

- Add/update market prices

- Toggle transaction statuses (Pending ↔ Complete)

- Delete transactions as needed

- Manage Bangalore-specific location assignments

- Change contract statuses in both directions (PENDING ↔ COMPLETED)

- Override automated contract completion status when needed

- Accept/reject contract requests on behalf of farmers

4. Market Prices

- View latest market prices for various commodities

- Check price history with interactive charts

- Filter by time period (daily, weekly, monthly)

- See price trends and geographic variations

- Compare prices across different locations

- **Bangalore Areas**: Track prices across 20 Bangalore-specific areas (Indiranagar, Koramangala, Whitefield, etc.)

- **Automated Date Updates**: Market price dates automatically update daily at midnight

- **Bilingual Interface**: Full support for English and Kannada languages throughout the application

5. Transactions

- Track all payments related to contracts

- View transaction status (Pending/Completed/Failed)

- Monitor total amounts and pending payments

- Record different payment types (Advance, Partial, Final, Refund)

- **Automated Cash Completion**: Cash payments are automatically marked as COMPLETED upon creation

- **Enhanced Buyer Visibility**: Buyers can see all transactions related to their contracts, not just those they created

- **Admin Features**: Administrators can now toggle transaction status between Pending and Complete, and delete transactions as needed

■ API Endpoints

Authentication

- `POST /api/auth/register` - Register new user

- `POST /api/auth/login` - Login user

- `GET /api/auth/me` - Get current user

- `POST /api/auth/logout` - Logout user

Contracts

- `GET /api/contracts` - Get all contracts (filtered by role)
- `GET /api/contracts/:id` - Get contract details
- `POST /api/contracts` - Create new contract (Farmer only)
- `PUT /api/contracts/:id` - Update contract
- `PATCH /api/contracts/:id/status` - Update contract status (only admins can change COMPLETED contracts to other statuses)
- `GET /api/contracts/user/:userId` - Get contracts for specific user

Contract Requests

- `GET /api/contract-requests` - Get all contract requests (filtered by role)
- `GET /api/contract-requests/:id` - Get single request details
- `POST /api/contract-requests` - Create new request (Buyer only)
- `PATCH /api/contract-requests/:id/accept` - Accept request (Farmer/Admin - creates contract)
- `PATCH /api/contract-requests/:id/reject` - Reject request (Farmer/Admin)
- `PATCH /api/contract-requests/:id/cancel` - Cancel request (Buyer only)

Ratings

- `GET /api/ratings/farmer/:farmerId` - Get farmer's ratings and average
- `POST /api/ratings` - Create rating for farmer (Buyer with completed contract)
- `PUT /api/ratings/:id` - Update own rating
- `DELETE /api/ratings/:id` - Delete rating (Admin or owner)

Market Prices

- `GET /api/market-prices` - Get market prices
- `GET /api/market-prices?latest=true` - Get latest prices
- `GET /api/market-prices/product/:productId` - Get price history for product
- `POST /api/market-prices` - Add market price (Admin/Buyer)
- `GET /api/market-prices/trends` - Get price trends

Products

- `GET /api/products` - Get all products
- `GET /api/products/:id` - Get product details
- `POST /api/products` - Create new product (Admin)
- `PUT /api/products/:id` - Update product (Admin)

Transactions

- `GET /api/transactions` - Get all transactions (filtered by user role - shows transactions for contracts user is part of as farmer/buyer)
- `GET /api/transactions/contract/:contractId` - Get transactions for contract

- `POST /api/transactions` - Create transaction (automatically updates contract to COMPLETED when all transactions complete)

- `PATCH /api/transactions/:id/status` - Update transaction status (automatically updates contract to COMPLETED when all transactions complete)

- `DELETE /api/transactions/:id` - Delete transaction (Admin only)

- `GET /api/transactions/user/:userId` - Get transactions for user

Users

- `GET /api/users` - Get all users (Admin only)

- `GET /api/users/by-role/:role` - Get users by role

- `GET /api/users/stats/:userId` - Get user statistics

- `PUT /api/users/:id` - Update user profile

■■ Database Options

Option A: PostgreSQL (Production)

Full relational database with Prisma ORM (Node.js) or JPA/Hibernate (Java) for production deployments.

Option B: JSON File Database (Development/Testing - Node.js Only)

All data stored in a single JSON file: `/server/config/database.json`

Data Storage:

- All application data in one portable file

- Automatic saving on every write operation

- Pre-loaded with sample data (users, products, contracts, prices)

- Easy backup and restore

- No database server required

- **Note**: Java backend uses PostgreSQL only

Sample Data Included:

- 3 Users (Admin, Farmer, Buyer)

- 5 Products (Tomato, Potato, Onion, Rice, Wheat)

- 1 Active Contract

- 3 Market Prices

- 1 Transaction

View Your Data:

```bash

# View all users

cat server/config/database.json | jq '.users'
```

```
# View entire database
cat server/config/database.json

# Or open in any text editor/JSON viewer
```

Backup Data:
```bash
cp server/config/database.json server/config/backup_$(date +%Y%m%d).json
```

Reset to Default:
```bash
rm server/config/database.json
npm run dev # Server recreates with sample data
```

For detailed information, see [`FILE_DATABASE_GUIDE.md`](server/FILE_DATABASE_GUIDE.md).

Main Entities (Both Modes)

- **User**: Stores farmer, buyer, and admin information with authentication
- **Product**: Agricultural commodities with categorization and units
- **Contract**: Structured agreements with lifecycle management
- **MarketPrice**: Real-time pricing with geographic and temporal data
- **Transaction**: Payment records with status tracking

Enum Definitions

- **UserRole**: `FARMER`, `BUYER`, `ADMIN`
- **ContractStatus**: `DRAFT`, `PENDING`, `ACTIVE`, `COMPLETED`, `CANCELLED`
- **TransactionStatus**: `PENDING`, `COMPLETED`, `FAILED`
- **PaymentType**: `ADVANCE`, `PARTIAL`, `FINAL`, `REFUND`, `OTHER`
- **ProductCategory**: `GRAINS`, `VEGETABLES`, `FRUITS`, `SPICES`, `PULSES`, `OTHERS`
- **ContractRequestStatus**: `PENDING`, `ACCEPTED`, `REJECTED`, `CANCELLED`

Contract Status Flow

```
DRAFT → PENDING → ACTIVE → COMPLETED
↓
```

CANCELLED
```

### Automatic Contract Completion

- Contracts automatically transition to **COMPLETED** status when all related transactions are marked as COMPLETED
- Admins retain full control and can change COMPLETED contracts back to other statuses
- Non-admin users cannot change COMPLETED contracts to other statuses

## ■■ Development

### Available Scripts

**Root directory**:
- `npm run dev` - Start both server and client in development mode
- `npm run build` - Build both server and client for production
- `npm run install:all` - Install dependencies for all packages

**Server directory**:
- `npm run dev` - Start backend server with hot reloading
- `npm run build` - Compile TypeScript to JavaScript
- `npm run start` - Start compiled server
- `npm run prisma:generate` - Generate Prisma client
- `npm run prisma:migrate` - Run database migrations
- `npm run prisma:studio` - Open Prisma Studio UI
- `npm run seed` - Seed database with sample data
- `npm run add-prices` - Add sample market prices
- `npm run add-contracts` - Add sample contracts
- `npm run add-transactions` - Add sample transactions
- `npm run analyze-prices` - Analyze market price trends
- `npm run update-prices` - Update market prices

**Client directory**:
- `npm run dev` - Start frontend development server
- `npm run build` - Build static assets for production
- `npm run preview` - Preview production build locally
- `npm run lint` - Lint TypeScript files

### Building for Production

```bash

```
# Build everything
npm run build

# Build server only
cd server
npm run build

# Build client only
cd client
npm run build
```

Database Management

PostgreSQL Mode:

```bash
# Create a new migration
cd server
npx prisma migrate dev --name migration_name

# Reset database (■■ deletes all data)
npx prisma migrate reset

# View database in browser
npm run prisma:studio

# Pull schema from database
npx prisma db pull

# Push schema to database
npx prisma db push
```

File Database Mode:

```bash
# View data
cat server/config/database.json

# Backup data
cp server/config/database.json server/config/backup.json

# Reset to default (delete file, server recreates)
```

rm server/config/database.json

# Import/Export data (programmatically)
const { dbFileManager } = require('./src/config/dbFileManager');
dbFileManager.exportAll('./backup.json');
dbFileManager.importAll('./backup.json');
```

### Environment Variables

**Server (.env)**:

```env
# PostgreSQL Mode (USE_FILE_DATABASE=false)
DATABASE_URL="postgresql://user:password@localhost:5432/contract_farming"
JWT_SECRET="your-secret-key"
JWT_EXPIRES_IN="7d"
PORT=5000
NODE_ENV=development
USE_FILE_DATABASE=false

# File Database Mode (USE_FILE_DATABASE=true)
USE_FILE_DATABASE=true
FILE_DATABASE_AUTO_SAVE=true
FILE_DATABASE_PATH=./config/database.json
FILE_DATABASE_BACKUP_PATH=./config/backup_all_data.json
```

■ Java Spring Boot Backend - Enterprise Migration ■ NEW

This project now includes a **complete Java Spring Boot backend** as an alternative to the Node.js implementation!

Why Java Backend?

- ■ **Enterprise-Grade**: Spring Boot ecosystem with production-ready features
- ■ **Type Safety**: Java's strong typing reduces runtime errors
- ■ **Better Performance**: JVM optimization and multithreading capabilities
- ■ **Enhanced Security**: Built-in Spring Security with JWT authentication
- ■ **Scalability**: Better suited for large-scale, high-traffic applications
- ■ **100% Compatible**: Works with the existing React frontend without changes

Java Backend Features
```

Completed Implementation:
- ■ Spring Boot 3.2.0 with Java 17
- ■ Spring Security with JWT authentication
- ■ JPA/Hibernate ORM for database operations
- ■ Complete REST API matching Node.js endpoints
- ■ All core entities (User, Product, Contract, MarketPrice, Transaction)
- ■ Data transfer objects (DTOs) for clean API design
- ■ CORS configuration for React frontend
- ■ Maven build automation
- ■ PostgreSQL database integration

Available Endpoints:
- Authentication: `/api/auth/login`, `/api/auth/register`, `/api/auth/me`
- Products: `/api/products`, `/api/products/{id}`
- Market Prices: `/api/market-prices`, `/api/market-prices/latest`
- Contracts: `/api/contracts`, `/api/contracts/{id}`
- Transactions: `/api/transactions`, `/api/transactions/{id}`
- Users: `/api/users`, `/api/users/{id}`, `/api/users/role/{role}`

Quick Start with Java

```bash
# Navigate to Java backend
cd server-java

# Build and run
mvn spring-boot:run

# Or use the convenience script
cd ..
./run-java-backend.sh
```

Switching Between Backends

To use Node.js backend:
```bash
npm run dev # Starts Node.js server on port 5000
```

To use Java backend:

```bash
./run-java-backend.sh # Starts Java server on port 5000
```

> **Important**: Both backends run on the same port (5000) and provide identical APIs. You can switch between them without changing any frontend code!

For detailed Java backend documentation, see
[`server-java/README.md`](server-java/README.md) and
[`server-java/MIGRATION_COMPLETE.md`](server-java/MIGRATION_COMPLETE.md).

■ Troubleshooting

Common Issues

Database Connection Issues (PostgreSQL Mode):

- Verify PostgreSQL is running
- Check DATABASE_URL in `server/.env`
- Ensure database exists: `CREATE DATABASE contract_farming;`
- Confirm PostgreSQL service is active

File Database Issues:

- Check `USE_FILE_DATABASE=true` in `.env`
- Ensure write permissions for `/server/config/` directory
- Verify `database.json` file exists and is valid JSON
- Delete `database.json` to reset to default sample data

Port Already in Use:

- Change PORT in `server/.env`
- Update proxy in `client/vite.config.ts` if needed
- Check for existing processes using the same port

Prisma Client Issues:

```bash
cd server
npm run prisma:generate
```

Module Not Found Errors:

```bash
# Reinstall dependencies
npm run install:all
```

Authentication Problems:

- Verify JWT_SECRET is set correctly
- Check that tokens are not expired
- Clear browser cache and cookies if needed

API Request Failures:

- Confirm server is running on specified port
- Check CORS settings in server configuration
- Verify API endpoint URLs

Development Tips

- Always run database migrations after schema changes (PostgreSQL mode)
- Use the seed script to populate sample data during development
- Check the browser console and server logs for detailed error messages
- Use Prisma Studio to inspect database contents (PostgreSQL mode)
- Use file database mode for quick setup and testing
- Backup `database.json` regularly when using file storage
- Switch between PostgreSQL and file database by toggling `USE_FILE_DATABASE`

■ Deployment (Vercel & Neon PostgreSQL)

This project is optimized for deployment on Vercel with a Neon Serverless PostgreSQL database.

1. Database Setup (Neon)

1. Go to [Neon.tech](https://neon.tech) and create a new project.
2. Get your connection string (e.g., `postgresql://user:pass@ep-xxx.neon.tech/neondb?sslmode=require`).
3. Add `&connection_limit=1` to the end of the URL to prevent connection limits on serverless environments.

2. Backend Deployment (Vercel)

1. Deploy the `server` directory to Vercel.
2. In your Vercel project settings, set the following Environment Variables:

- `DATABASE_URL`: Your Neon connection string.
- `USE_FILE_DATABASE`: `false` (Required for production).
- `JWT_SECRET`: A strong random string for authentication.
- `JWT_EXPIRES_IN`: `7d`
- `NODE_ENV`: `production`
- `CLIENT_URL`: URL of your frontend (e.g., `https://client-opal-beta-80.vercel.app`)

3. Database Migration
Run migrations against your production database from your local machine:
```bash
cd server
export DATABASE_URL="your-neon-connection-string"
npm run prisma:generate
npm run prisma:migrate
npm run seed # Optional: Seed initial admin user and data
```

4. Frontend Deployment (Vercel)
1. Deploy the `client` directory to Vercel.
2. Set the Environment Variable in Vercel settings:
- `VITE_API_URL`: URL of your deployed backend (e.g., `https://server-theta-lime-34.vercel.app`)

Current Live Deployment Links
- **Frontend**: https://client-opal-beta-80.vercel.app
- **Backend API**: https://server-theta-lime-34.vercel.app

■ Contributing

We welcome contributions to enhance the agricultural commodity trading platform!

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Make your changes
4. Add tests if applicable
5. Test thoroughly
6. Commit your changes (`git commit -m 'Add amazing feature'`)
7. Push to the branch (`git push origin feature/amazing-feature`)
8. Open a pull request

Development Guidelines

- Follow the existing code style and patterns
- Write clear, descriptive commit messages
- Update documentation as needed
- Add tests for new functionality
- Ensure all tests pass before submitting

■ Additional JavaScript Utilities

This project includes comprehensive JavaScript utility modules:

- **General Utilities** (`jsUtils.js`): Currency formatting, mathematical calculations, validations
- **Market Analysis** (`marketPriceAnalysis.js`): Price trends, volatility, predictions
- **Contract Management** (`contractUtils.js`): Status tracking, completion calculations
- **Chart Visualization** (`chartUtils.js`): Data preparation for charts
- **Localization** (`localization.js`): Multi-language support (English/Kannada)
- **Notifications** (`notificationUtils.js`): Toasts, dialogs, loading spinners
- **Validation** (`validationUtils.js`): Form and input validation
- **Data Analytics** (`dataAnalytics.js`): Statistical calculations and analysis
- **Date/Time Operations** (`dateUtils.js`): Date formatting and manipulation
- **API Utilities** (`apiUtils.js`): Request handling and error management

These utilities can be imported individually or as a complete suite via the main index file.

■ New Features (Latest Updates)

■ Java Spring Boot Backend Migration ■ NEWEST
- **Complete backend migration** from Node.js to Java Spring Boot 3.2.0
- **Enterprise-grade architecture** with Spring Security and JPA/Hibernate
- **100% frontend compatibility** - React app works without any changes
- **Enhanced performance** with JVM optimization and multithreading
- **Production-ready** with built-in security and scalability features
- **Maven build system** for streamlined dependency management
- **Location**: `server-java/` directory with full documentation

■ Automated Cash Payment Completion
- Cash payments are **automatically marked as COMPLETED** when created
- Other payment methods (Bank Transfer, UPI, Card, etc.) remain PENDING for verification
- Streamlines cash transaction workflow
- No manual status update needed for cash payments
- **Location**: `server/src/routes/transactions.ts`

■ Admin Permanent User Deletion
- Admins can **permanently delete users** with complete data cleanup
- Deletes all associated data in a safe transaction:
- User account
- All contracts (as buyer or farmer)
- All transactions
- All ratings (given and received)
- All notifications (sent and received)

- All contract requests
- **Double confirmation** required to prevent accidental deletion
- Admin users cannot be deleted
- Admins cannot delete themselves
- **Location**: `server/src/routes/users.ts`, `client/src/pages/Users.tsx`

■ Automated Bangalore Area Date Updates

- **20 Bangalore areas** with automated daily date updates
- Areas include: Indiranagar, Koramangala, Whitefield, HSR Layout, BTM Layout, Jayanagar, Malleshwaram, Electronic City, Marathahalli, Bannerghatta, Hebbal, Yelahanka, Frazer Town, RT Nagar, Peenya, Banashankari, Basavanagudi, Wilson Garden, Ulsoor, KR Puram
- **Runs daily at midnight** - updates all market price dates to current date
- **Preserves price values** - only updates dates, not prices
- Prevents duplicate entries for the same day
- **Location**: `server/src/updateBangaloreAreaDates.ts`

■ Glass Morphism Login UI

- Modern **transparent glass-effect** login interface
- Frosted glass card with backdrop blur
- Beautiful overlay on background image
- Enhanced visual appeal with drop shadows
- All form elements styled with glass effect
- **Location**: `client/src/pages/Login.tsx`

■ License

MIT License - Feel free to use, modify, and distribute this software.

■ Key Highlights

- **Dual Backend Support**: Choose between Node.js/Express or Java Spring Boot
- **Frontend Compatibility**: Same React UI works with both backends
- **Production Ready**: Both backends support PostgreSQL for production
- **Easy Development**: JSON file database for quick testing (Node.js only)
- **Enterprise Features**: Java backend with Spring Security and JPA
- **Rapid Development**: Node.js with hot reloading and Prisma ORM
- **Cloud Deployment**: Ready for Vercel, Railway, or any cloud platform
- **Bilingual Support**: Full English and Kannada language support
- **Modern UI**: Glass morphism design with responsive layout

■ Support

For issues, questions, or suggestions:

- Create an issue in the GitHub repository
- Contact the development team
- Check the documentation for common solutions

Empowering farmers and buyers with transparent, efficient agricultural trade